

ALGORITMOS - RECURSIVIDADE - C#

Tiago Henrique Angioletti





O QUE É RECURSIVIDADE?

Recursividade é uma técnica de programação onde uma função chama a si mesma para resolver um problema. Essa abordagem é útil para dividir problemas complexos em subproblemas menores e mais gerenciáveis.



COMO FUNCIONA A RECURSIVIDADE?

Toda função recursiva precisa de dois elementos fundamentais:

CASO BASE: Condição que encerra a recursão para evitar loops infinitos.

CHAMADA RECURSIVA: A função chama a si mesma com um argumento modificado, aproximando-se do caso base.



EXEMPLO DE RECURSIVIDADE: FATORIAL

O cálculo do fatorial de um número é um exemplo clássico de recursividade. O fatorial de n (representado por $n!$) é definido como:

$$n! = n * (n - 1)!$$

Com o caso base: $0! = 1$

IMPLEMENTAÇÃO EM C#



```
using System;

class Program
{
    static int Fatorial(int n)
    {
        if (n == 0)
            return 1; // Caso base
        return n * Fatorial(n - 1); // Chamada recursiva
    }

    static void Main()
    {
        Console.WriteLine($"Fatorial de 5: {Fatorial(5)}");
    }
}
```



Saída esperada:

Fatorial de 5: 120



VANTAGENS E DESVANTAGENS DA RECURSIVIDADE

VANTAGENS:

SIMPLICIDADE: Facilita a compreensão e a escrita de código para problemas complexos.

DIVISÃO DE PROBLEMA: Permite resolver problemas de forma incremental.

DESVANTAGENS:

CONSUMO DE MEMÓRIA: Cada chamada recursiva ocupa uma nova posição na pilha de execução.

RISCO DE STACK OVERFLOW: Se não for bem implementada, pode levar a um grande número de chamadas, causando estouro de pilha.



CONCLUSÃO

Recursividade é uma técnica poderosa, mas deve ser usada com cuidado para evitar problemas de desempenho. Sempre que possível, avalie se uma solução iterativa ou memoização pode ser mais eficiente.